ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT TECHNIQUE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

ENSET

المدرسة العليا لأساتذة التعليم التقني
المحمدية
جامعة الحسن الثاني بالدار البيضاء

Atelier 4 – MLOps de bout en bout
DVC + S3 + GitHub Actions + CML

Projet : `churn-cml-dvc`
Pr. Soufiane HAMIDA

24 janvier 2026

Objectif du workshop. À la fin de ce TP, vous aurez :

- un bucket S3 où stocker vos datasets et modèles,
- un utilisateur IAM dédié avec des permissions minimales,
- un projet Machine Learning versionné avec **DVC** (Data Version Control),
- un pipeline CI/CD **GitHub Actions** qui :
  - récupère les données depuis S3,
  - entraîne le modèle,
  - pousse les artefacts (data & modèles) dans S3 avec DVC,
  - publie automatiquement un rapport (métriques + matrice de confusion) en commentaire sur GitHub grâce à **CML**.

Table des matières

1 Prérequis techniques **2**
1.1 Outils nécessaires côté étudiant . . . 2
1.2 Versions de référence du TP . . . 2

2 Étape 1 – Préparer AWS **2**
2.1 1.1 Créer un bucket S3 . . . 2
2.2 1.2 Créer un utilisateur IAM technique . . . 3
2.3 1.3 Attacher une policy minimale S3 . . . 3
2.4 1.4 Récupérer les credentials IAM . . . 4

3 Étape 2 – Initialiser DVC en local **4**
3.1 2.1 Installer les dépendances Python du projet . . . 4
3.2 2.2 Initialiser DVC dans le repo . . . 4
3.3 2.3 Déclarer le remote S3 dans DVC . . . 4
3.4 2.4 Stocker les credentials **en local uniquement** . . . 4
3.5 2.5 Versionner `data/` et `models/` avec DVC . . . 5

4 Étape 3 – Préparer GitHub **5**
4.1 3.1 Pousser le code sur GitHub . . . 5
4.2 3.2 Définir les secrets GitHub . . . 6

5 Étape 4 – Pipeline GitHub Actions (CI/CD MLOps) **6**
5.1 4.1 Le workflow `.github/workflows/ci.yml` . . . 6
5.2 4.2 Points importants à expliquer aux étudiants . . . 7

6 Étape 5 – Résumé du flux de bout en bout 8

1 Prérequis techniques

1.1 Outils nécessaires côté étudiant

— Un compte GitHub.
— Un compte AWS (même niveau *Free Tier* suffit).
— Git et Python installés en local.
— DVC installé en local (sinon on l'installe plus tard).

1.2 Versions de référence du TP

Pour garantir la reproductibilité :

— Python : 3.11
— NumPy : 2.x
— scikit-learn : ≥ 1.7, < 1.8
— imbalanced-learn : ≥ 0.14, < 0.15
— DVC : 3.63.0 avec support S3

Nous utiliserons un projet d'exemple `churn-cml-dvc` contenant :

— un script `script.py` qui entraîne un modèle (classification client churn),
— un dossier `data/` (features d'entraînement, etc.),
— un dossier `models/` (modèle entraîné, métriques),
— un workflow CI/CD GitHub Actions.

2 Étape 1 – Préparer AWS

Cette section couvre la création d'un bucket S3 pour stocker les artefacts ML, et d'un utilisateur IAM *machine* avec des permissions minimales (principe de moindre privilège).

2.1 1.1 Créer un bucket S3

1. Connectez-vous à la console AWS.
2. Recherchez "S3" dans la barre de recherche et ouvrez le service.
3. Cliquez sur `Create bucket`.
4. Donnez un nom globalement unique, par exemple :

```
my-bucket-dvc-soufiane
```

5. Sélectionnez la région. Exemple utilisé dans ce TP :

```
eu-west-3   # Paris
```

6. Laissez le contrôle d'accès par défaut (le bucket reste privé).
7. Validez avec `Create bucket`.

Nous allons utiliser le chemin interne suivant pour DVC :

```
s3://my-bucket-dvc-soufiane/dvc-storage
```

On crée souvent un préfixe dédié, ici `dvc-storage`, pour ne pas mélanger avec d'autres fichiers S3 éventuels.

2.2 1.2 Créer un utilisateur IAM technique

Cet utilisateur sera utilisé par :

— **votre machine locale** (quand vous faites `dvc push` / `dvc pull`),

— **GitHub Actions** (pipeline CI/CD).

Étapes :

1. Ouvrir le service `IAM` dans la console AWS.
2. Aller dans `Users` → `Create user`.
3. Nommer l'utilisateur, par ex. :

```
dvc-ci-user
```

4. Cocher `Access key - Programmatic access` (accès programmatique uniquement).
5. Valider jusqu'à l'étape des permissions (on va créer une policy custom juste après).

2.3 1.3 Attacher une policy minimale S3

On veut donner à cet utilisateur seulement le droit de :

— lister le contenu du bucket dans le dossier `dvc-storage/`,

— lire / écrire / supprimer des objets **dans ce dossier seulement**.

Voici un exemple de policy JSON (IAM → `Policies` → `Create policy` → onglet JSON) :

Listing 1 – Policy IAM minimale pour DVC sur un répertoire S3 dédié.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowListBucket",
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket"
            ],
            "Resource": "arn:aws:s3:::my-bucket-dvc-soufiane",
            "Condition": {
                "StringLike": {
                    "s3:prefix": [
                        "dvc-storage/*"
                    ]
                }
            }
        },
        {
            "Sid": "AllowBucketObjectsCRUD",
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:PutObject",
                "s3:DeleteObject"
            ],
            "Resource": "arn:aws:s3:::my-bucket-dvc-soufiane/dvc-storage/*"
        }
    ]
}
```

Validez la policy, puis attachez-la à l'utilisateur `dvc-ci-user`.

2.4 1.4 Récupérer les credentials IAM

Une fois l'utilisateur créé, AWS vous affiche :

— `Access key ID` (ex : `AKIA...`),

— `Secret access key` (longue chaîne).

Important : notez ces deux valeurs immédiatement. Elles serviront :

— en local (variables d'environnement ou fichier DVC local),

— dans GitHub Secrets.

3 Étape 2 – Initialiser DVC en local

Ici on configure DVC pour qu'il envoie les données et les modèles dans le bucket S3.

3.1 2.1 Installer les dépendances Python du projet

Dans votre environnement local (par exemple `venv` ou `conda`), installez les dépendances ML. On utilise un `requirements.txt` propre comme ci-dessous :

Listing 2 – Exemple de requirements.txt propre pour ce projet.

```
numpy>=2,<3
scipy>=1.11,<2
pandas>=2,<3

matplotlib>=3.7
seaborn>=0.12

scikit-learn>=1.7,<1.8
imbalanced-learn>=0.14,<0.15
joblib>=1.4
```

Installez ensuite DVC avec support S3 :

```
pip install "dvc[s3]==3.63.0"
```

3.2 2.2 Initialiser DVC dans le repo

Dans le dossier racine du projet (là où se trouve votre `.git`) :

```
dvc init
git add .dvc .gitignore
git commit -m "Init␣DVC"
```

3.3 2.3 Déclarer le remote S3 dans DVC

On dit à DVC : "mon stockage distant (remote) est un bucket S3".

```
dvc remote add -d myremote s3://my-bucket-dvc-soufiane/dvc-storage
dvc remote modify myremote region eu-west-3
```

Note :

— `-d` signifie "c'est le remote par défaut".

— `region eu-west-3` doit correspondre à la région réelle du bucket.

3.4 2.4 Stocker les credentials en local uniquement

DVC peut lire vos identifiants AWS de deux façons :

Option A : Variables d'environnement (recommandé)

```
set AWS_ACCESS_KEY_ID=AKIA...
set AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxxxxxx
set AWS_DEFAULT_REGION=eu-west-3
```

(équivalent PowerShell/Windows ou `export` sous Linux/Mac).

Option B : Fichier local non versionné

```
dvc remote modify myremote access_key_id AKIA...
dvc remote modify myremote secret_access_key "xxxxxxxxxxxxxxxx"
```

Ces infos sont écrites dans `.dvc/config.local`, **fichier qui ne doit jamais partir sur Git**. On le retire explicitement de Git :

```
git rm --cached .dvc/config.local
echo ".dvc/config.local" >> .gitignore
git commit -m "Do␣not␣track␣DVC␣local␣credentials"
```

3.5 2.5 Versionner `data/` et `models/` avec DVC

Supposons que `script.py` génère :

- les données préparées dans `data/`,
- le modèle entraîné et les métriques dans `models/`

On lance d'abord l'entraînement local :

```
python script.py
```

Ensuite on dit à DVC de suivre ces répertoires :

```
dvc add data models
git add data.dvc models.dvc
git commit -m "Track␣data/␣and␣models/␣with␣DVC"
```

Puis on pousse les blobs réels vers S3 :

```
dvc push
```

Après `dvc push`, le bucket S3 contient les objets correspondant à vos jeux de données et vos modèles versionnés (hashés par DVC). C'est cette étape qui "bootstrape" le remote S3. Sans elle, un `dvc pull` sur une autre machine échouera avec "missing cache files".

4 Étape 3 – Préparer GitHub

4.1 3.1 Pousser le code sur GitHub

Classique :

```
git remote add origin git@github.com:<votre-user>/<churn-cml-dvc>.git
git push -u origin main
```

Le repo doit contenir au minimum :

- `script.py` (entraîne le modèle, génère `metrics.txt`, `conf_matrix.png`, etc.)
- `requirements.txt` (les libs ML, **pas** DVC ni CML dedans)
- `data.dvc`, `models.dvc` (métadonnées DVC, mais pas les gros fichiers)
- `.dvc/config` (le remote S3, sans les secrets)
- `.github/workflows/ci.yml` (pipeline CI/CD)

4.2 3.2 Définir les secrets GitHub

Dans GitHub :

1. Aller dans `Settings` du repo.
2. Aller dans `Secrets and variables` → `Actions`.
3. Ajouter les secrets suivants :

— `AWS_ACCESS_KEY_ID` = la clé IAM `Access key ID`

— `AWS_SECRET_ACCESS_KEY` = la clé IAM `Secret access key`

— `AWS_DEFAULT_REGION` = `eu-west-3` (ou votre région)

Note : `GITHUB_TOKEN` est déjà fourni automatiquement par GitHub Actions. On l'utilisera sous le nom `${{ secrets.GITHUB_TOKEN }}` pour publier le rapport CML.

Jamais committer les clés AWS dans le dépôt.

5 Étape 4 – Pipeline GitHub Actions (CI/CD MLOps)

L'objectif du pipeline :

1. Installer l'environnement Python (avec scikit-learn, imbalanced-learn, etc.).
2. Installer DVC + plugin S3.
3. Récupérer les données du dernier run via `dvc pull`.
4. Lancer `script.py` pour ré-entraîner.
5. Versionner `data/` et `models/` mis à jour avec DVC.
6. Pousser les nouveaux artefacts vers S3 via `dvc push`.
7. Générer un rapport et le poster en commentaire dans le pull request via **CML**.

5.1 4.1 Le workflow `.github/workflows/ci.yml`

Listing 3 – Exemple de workflow GitHub Actions complet.

```
name: churn-imbalanced-dataset
on: [push]
permissions: write-all

jobs:
  run:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repo
        uses: actions/checkout@v3

      - name: Set up Python 3.11
        uses: actions/setup-python@v4
        with:
          python-version: "3.11"

      - name: Set up CML
        uses: iterative/setup-cml@v2

      - name: Install Python deps
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          AWS_DEFAULT_REGION: ${{ secrets.AWS_DEFAULT_REGION }}
        run: |
          python -m pip install --upgrade pip
```

```
        pip install -r requirements.txt
        pip install "dvc[s3]==3.63.0"

    - name: DVC pull (restore data/models depuis S3)
      env:
        AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        AWS_DEFAULT_REGION: ${{ secrets.AWS_DEFAULT_REGION }}
      run: |
        dvc pull || echo "No remote cache yet"

    - name: Train / Eval / Push artifacts
      env:
        AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        AWS_DEFAULT_REGION: ${{ secrets.AWS_DEFAULT_REGION }}
      run: |
        # 1. Entrainement local du modèle
        python script.py
        # script.py doit produire:
        #   - metrics.txt
        #   - conf_matrix.png
        #   - dossiers data/ et models/ mis à jour

        # 2. Versionner les outputs data/ et models/ avec DVC
        dvc add data models

        # 3. Committer les .dvc générés
        git config user.email "github-actions[bot]@users.noreply.github.com"
        git config user.name "github-actions[bot]"

        git add data.dvc models.dvc || true
        [ -f dvc.lock ] && git add dvc.lock

        git commit -m "Update data/models [skip ci]" || echo "nothing to
            commit"

        # 4. Pousser les blobs vers S3
        dvc push

    - name: Publish CML report
      env:
        REPO_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      run: |
        echo "## Metrics" > report.md
        awk 'NR%4{print "    " $0}{print ""}' metrics.txt >> report.md

        echo "## Confusion Matrix" >> report.md
        echo "![](./conf_matrix.png)" >> report.md

        # commentaire automatique sur le commit / PR
        cml comment create --pr report.md
```

5.2 4.2 Points importants à expliquer aux étudiants

— **setup-python** force l'environnement Python (ici 3.11).
Cela garantit la compatibilité scikit-learn / numpy.

— **setup-cml@v2** installe l'outil CML (en Node.js).
Pas besoin d'ajouter un paquet Python `cml`.

— On installe `dvc[s3]` séparément, **après** les libs ML.
Ça évite un conflit de dépendances trop complexe pour pip.
— Les variables `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION` viennent des secrets GitHub.
— `dvc pull` au début du job permet de récupérer la dernière version connue des données/modèles depuis S3.
— Après l'entraînement, on `dvc add` les répertoires `data/` et `models/`, puis on `dvc push` pour envoyer la nouvelle version vers S3.
— On commit les fichiers `data.dvc`, `models.dvc` (et éventuellement `dvc.lock`) directement depuis la CI. Cela met à jour GitHub avec les métadonnées de la nouvelle version des artefacts.
— Enfin, on génère un `report.md` avec les métriques (`metrics.txt`) et une image de matrice de confusion (`conf_matrix.png`), puis on le publie via CML en commentaire GitHub.

6 Étape 5 – Résumé du flux de bout en bout

Pour clôturer le workshop, voici la **chaîne complète** qu'on attend après implémentation :

1. **Sur AWS**
   (a) Créer un bucket S3 (ex : `my-bucket-dvc-soufiane`, région `eu-west-3`)
   (b) Créer un utilisateur IAM technique `dvc-ci-user`
   (c) Attacher une policy JSON minimale pour autoriser l'accès uniquement à `dvc-storage/` dans le bucket.
   (d) Récupérer `Access key ID` et `Secret access key`.
2. **En local (dev)**
   (a) Installer l'environnement Python (requirements ML).
   (b) `dvc init`, puis `dvc remote add ... s3://.../dvc-storage`.
   (c) Configurer les credentials AWS localement (variables d'env ou `.dvc/config.local` non versionné).
   (d) Lancer `script.py` pour générer `data/`, `models/`, `metrics.txt`, `conf_matrix.png`.
   (e) `dvc add data models`, `dvc push` : les artefacts partent dans S3.
   (f) `git commit` & `git push` : le code + les fichiers `.dvc` montent sur GitHub.
3. **Sur GitHub**
   (a) Ajouter les secrets (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION`).
   (b) Ajouter le workflow GitHub Actions `ci.yml`.
   (c) Chaque push déclenche le pipeline :
   — Installe Python + dépendances.
   — `dvc pull` : récupère la dernière version des données et modèles depuis S3.
   — Exécute `script.py` : ré-entraînement.
   — `dvc add`, `git commit`, `dvc push` : publie la nouvelle version des artefacts vers S3.
   — Génére un `report.md` avec les métriques et la matrice de confusion.
   — `cml comment create -pr report.md` : publie les résultats dans GitHub.

Ce flux réalise un cycle MLOps complet :
— **versioning du code** (Git),
— **versioning des données et modèles** (DVC + S3),
— **reproductibilité de l'entraînement** (CI),
— **traçabilité des performances** (rapport automatique CML).

N.B
Vous venez de mettre en place une vraie boucle MLOps : chaque changement de code relance l'entraînement, pousse les artefacts, et publie un rapport traçable. C'est le socle d'une plateforme ML fiable.